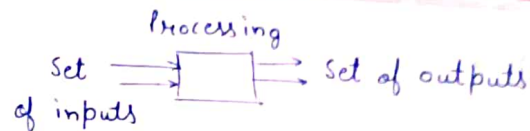


Theory of Computation (Automata Theory)



Date 01-08-2023

It is a theoretical branch of CSE & Maths, which mainly deals with the logic of computation with respect to simple machines, referred to as automata.

It enables to understand how machines compute the functions & solve them.

Main motivation - To develop methods to describe & analyze the dynamic behaviour of discrete systems.

► Symbol - Also called a character, is the smallest building block, which can be any alphabet, letter or picture.

example - a, b, c, 0, 1, ...

► Alphabets (Σ) - These are set of symbols, which are always finite.

$\Sigma = \{0, 1\}$ is an alphabet of binary digits

$\Sigma = \{0, 1, \dots, 9\}$ is an alphabet of decimal digits

$\Sigma = \{a, b, c\}$

$\Sigma = \{A, B, C, \dots, Z\}$

► string (w) - It is a finite sequence of symbols from some alphabets.

example -

If $\Sigma = \{a, b\}$, various strings that can be generated from Σ are

$\{ab, aa, aaa, bb, bbb, ba, aba, \dots\}$

Empty String - String with zero occurrences of symbols (ϵ)

Length of string - Number of symbols in a string (w) denoted by $|w|$

$$w = 010$$

$$\therefore |w| = 3$$

Note : If the no. of symbols in alphabet Σ is represented by $|\Sigma|$, then number of strings of length n possible over $\Sigma = |\Sigma|^n$

$\{a, b\}$ with length n $\therefore$ No. of strings generated $= 2^n$

Power of Σ : Σ^n

example : $\Sigma = \{a, b\}$ $\Sigma^2 = \Sigma \Sigma = \{a, b\}\{a, b\}$ $\Sigma^3 = \Sigma \Sigma \Sigma = \{a, b\}\{a, b\}\{a, b\}$

Reverse (w^R) : If $w = abb$ then $w^R = bba$

Date _____

* Closure Representation in TOC

L^+ : It is a Positive Closure that represents a set of all strings except Null strings (ϵ)

L^* : It is a "Kleene Closure", that represents the occurrence of certain alphabets for given language alphabets from zero to ∞ number of times. In which (ϵ) is also included.

$$L^* = \epsilon L^+$$

example : Regular expression for language accepting all combinations of g's over $\Sigma = \{g\}$

$$R = g^*$$

$$\therefore R = \{\epsilon, g, gg, ggg, gggg, \dots\}$$

$$R = g^+$$

$$\therefore R = \{g, gg, ggg, gggg, \dots\}$$

Σ^* is a set of all possible strings. It is called "Kleene star", "Kleene operator" or "Kleene closure".

► Language - Set of strings chosen from some Σ^* or we say that language is a subset of Σ^*

A language that can be formed over ' Σ ' can be finite or infinite

$$L_1 = \{\text{set of string of } \Sigma\}$$

$$L_1 = \{\text{set of all strings starting with 'b'}\}$$

$$L_1 = \{xy, yx, xx, yy\}$$

$$L_1 = \{babb, bab, baa, ba, \dots\}$$

* Finite Automata

- Used to recognize patterns
- It takes string of symbol as input & changes its state accordingly. When desired symbol is found, then transition occurs.

Concatenation $w = w_1 \cdot w_2$ [$w_1 = 1100$, $w_2 = 101$, $w = w_1 \cdot w_2 = 1100101$]

$w = 01011$

Prefix $\swarrow$ $\searrow$ Suffix

{ λ , 0, 01, 010, 0101, 01011 }

{ λ , 1, 11, 011, 1011, 01011 }

Date _____

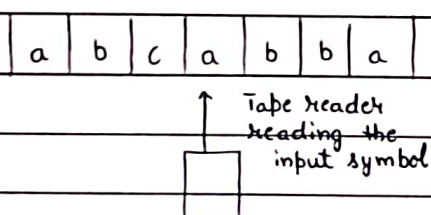
- At the time of transition, the automata can either move to next state or stay in the same.
- Finite Automata have 2 states $\begin{cases} \rightarrow \text{Accept state} \\ \rightarrow \text{Reject state} \end{cases}$

Definition of Finite Automata

A finite automaton is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where :

- Q is a finite non-empty set of states $\rightarrow q_0, q_1, q_2$
- Σ is a finite non-empty set of inputs called input alphabet
 $\Sigma = \{a, b\}$ or $\{0, 1\}$
- δ is a function which maps $Q \times \Sigma \rightarrow Q$ & it is called direct transition function.
This funcⁿ describes the change of state during the transition.
This mapping is represented by transition table or diagram.
- $q_0 \in Q$ is the starting state
- $F \subseteq Q$ is the set of final state.

* Finite Automata Model



Input Tape - linear tape having some number of cells.
Each input symbol is placed in each cell

Finite control - Decides the next state on receiving particular input from input tape.

Finite Automata

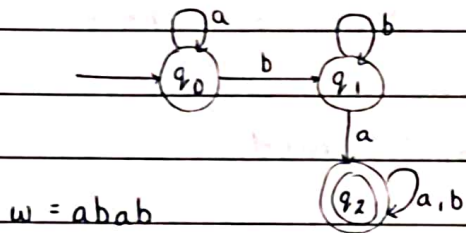
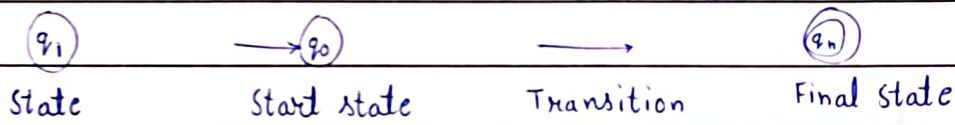
Deterministic FA

Non deterministic FA

- Uniqueness of computation
- In DFA, machine goes to one state only for a particular input character
- Doesn't accept null move.
- Used to transmit any number of states for a particular input
- Accept the null move.

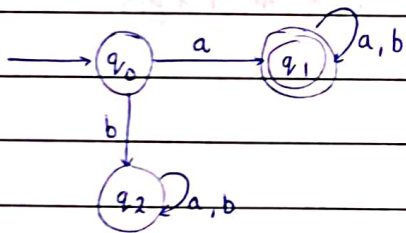
* Transition Diagram

A directed graph which can be constructed as :



Questions : Consider with minimum number of states

i. $\rightarrow$ String starting with a

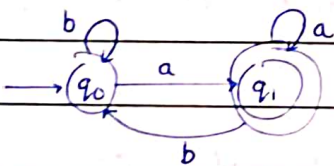


q_0 - initial state

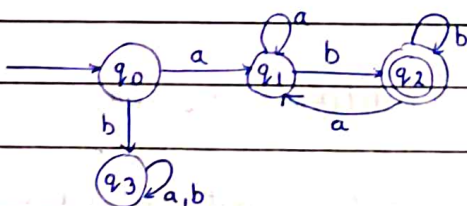
q_1 - final state

q_2 - dead state

ii. $\rightarrow$ String ending with a



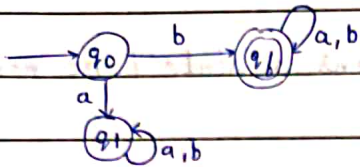
iii. $\rightarrow$ Starting with 'a' & ending with 'b'



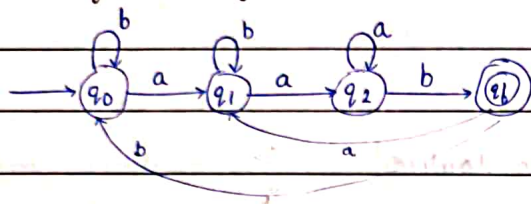
$L = \{ab, abab, ababab, aaabbb\}$

Date _____

iv. → All strings not starting with 'a'



v. → All strings ending with 'aab'



vi. →

* Transition Table

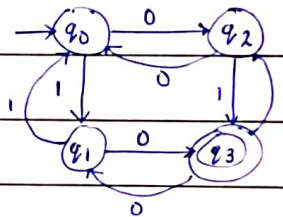
- A tabular representation of transition function.
- Takes 2 arguments (a state & a symbol) & returns a state (the next state)

Representation

- i → Columns correspond to input symbols
- ii → Rows correspond to states
- iii → Entries " " next states
- iv → Start state denoted by an arrow with no source
- v → The final state denoted by star or circle.

example :

State	Input	
	0	1
→ q ₀	q ₂	q ₁
q ₁	q ₃	q ₀
q ₂	q ₀	q ₃
q ₃	q ₁	q ₂

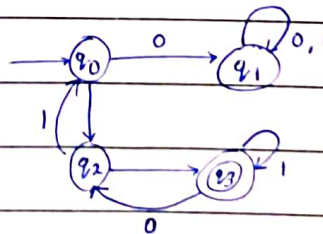


$\delta(q_0, 110101)$ Rejected

Note : If we reach final state after traversing complete string → Accepted.

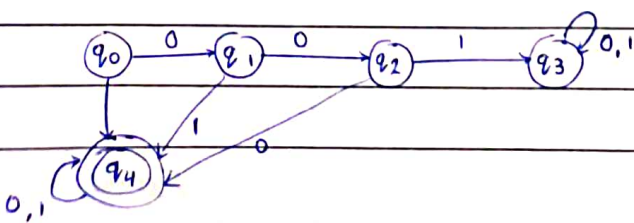
example :

State	Input	
	0	1
→ q ₀	q ₁	q ₂
q ₁	q ₁	q ₁
q ₂	q ₃	q ₀
q ₃	q ₂	q ₃

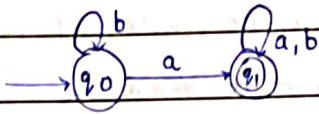


$\delta(q_0, 1101001001) \rightarrow$ Not Accepted

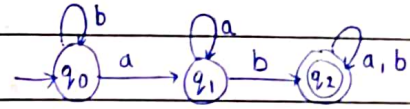
Q → Not starting with 001 or aab



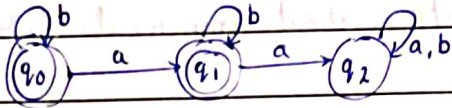
Q. → String containing atleast one 'a'



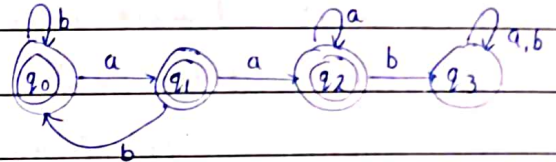
Q. → String with substring 'ab'



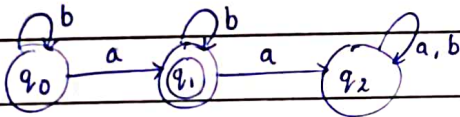
Q. → String containing atmost one 'a'



Q. → String not containing 'aab'



Q. → String containing exactly one 'a'

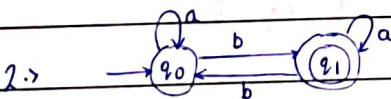
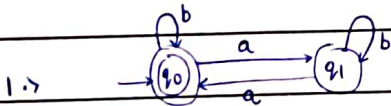
For

Q. → Strings containing even number of a's

Q. → odd number of b's

Q. → even number of a & odd number of b

Q. → even number of a or odd number of b



* DFA (Deterministic Finite Automata)

- Deterministic refers to the uniqueness of computation. The finite automata are called deterministic finite automata if the machine reads an input string one symbol at a time.
- There is only 1 path for specific input from current state to next state.
- Doesn't accept null move (can't change state without any input character).
- Can contain multiple final states.

Application - In Lexical Analysis in Compiler.

Formal Definition

A DFA is a collection of 5-tuples

Q : finite set of states

Σ : finite set of input symbols

q_0 : initial state

F : final state

δ : Transition function

Transition funcⁿ can be defined as : $\delta : Q \times \Sigma \rightarrow Q$

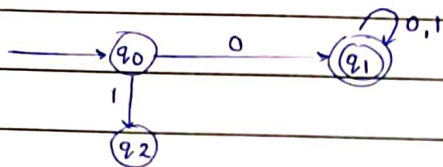
example 1 : $Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$ $q_0 = \{q_0\}$ $F = \{q_2\}$



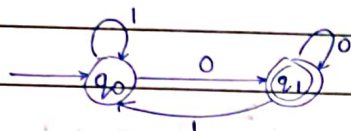
Transition Table

Present state	Next state for input 0	Next state of Input 1
$\rightarrow q_0$	q_0	q_1
q_1	q_2	q_1
q_2	q_2	q_2

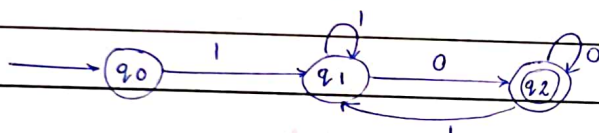
example : DFA with $\Sigma = \{0, 1\}$ accepts all starting with 0



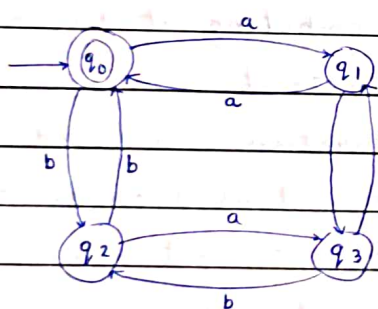
example : DFA with $\Sigma = \{0, 1\}$ accepts all ending with 0.



example : DFA with $\Sigma = \{0, 1\}$ accepts those strings which start with 1 & end with 0



example : DFA with $\Sigma = \{0, 1\}$ accepts even number of 0's & even number of 1's



q_0 - even a, even b

q_1 - odd a, even b

q_2 - Even a, odd b

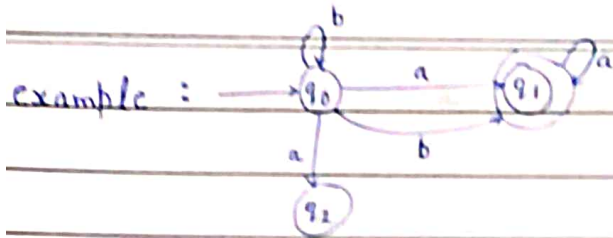
q_3 - odd a, odd b

★ Non deterministic Finite Automation (NDEFA or NEFA)

- Easy to construct than DFA
- The finite automata are called NEFA when there exist many paths for specific input from current state to the next state

★★★ Every NEFA is not DFA, but each NEFA can be translated into DFA.

- Defined in same way as DFA but with 2 exceptions, it contains multiple next states & it contains ϵ transition.



Formal Definition

Described by 5 tuple $(Q, \Sigma, \delta, q_0, F)$ where

Q - Set of states

Σ - Input alphabets

δ - Transition function mapping from $Q \times \Sigma \rightarrow 2^Q$ (power set of Q)
 set of all subsets of Q .

q_0 - Initial state

F - Set of final state.

★

DFA

- Can't use Empty string transition
- Can be understood as one machine
- Difficult to construct
- Dead configuration is not allowed
 (If we give input as 0 on q_0 state so we must give 1 as input to q_0 as self loop)

• $\delta : Q \times \Sigma \rightarrow Q$
 next possible state belongs to Q .

• All DFA are NFA

• $\delta(q_0, a) = q_1$

• Requires more space

NEA

- Can use empty string transition
- Can be understood as multiple little machines computing at same time.
- Easier to construct
- Dead configuration is allowed
 (If we give input as 0 on q_0 state so we can give next input 1 on q_1 which will go to next state)

• $\delta : Q \times (\Sigma \cup \epsilon) \rightarrow 2^Q$
 Next possible state belongs to power set of Q .

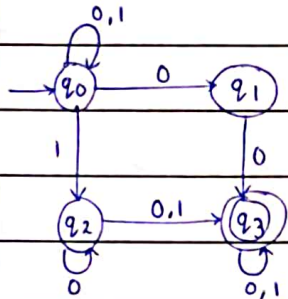
• Not all NEA are DFA

• $\delta(q_0, a) = \{q_1, q_2\}$

• Requires less space.

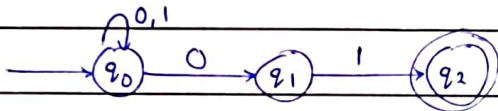
examples of NFA :

1. Design NFA for Present state



Present state	0	1
$\rightarrow q_0$	q_0, q_1	q_0, q_2
q_1	q_3	ϵ
q_2	q_2, q_3	q_3
q_3	q_3	q_3

2. NFA with $\Sigma = \{0, 1\}$ accepts all string ending with 01



* Equivalence of DFA & NFA

Let $M = (Q, \Sigma, \delta, q_0, F)$ is NFA which accepts language $L(M)$. There should be equivalent DFA denoted by $M' = (Q', \Sigma', \delta', q_0', F')$, such that $L(M) = L(M')$.

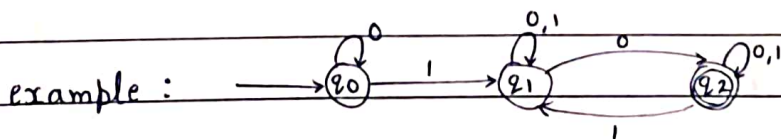
Steps for converting NFA into DFA

Step 1: Initially $Q' = \phi$

Step 2: Add q_0 of NFA to Q' . Then find the transitions from this start state

Step 3: In Q' , find possible set of states for each input symbol. If this set of states is not in Q' , then add it to Q' .

Step 4: In DFA, the final states will be the states which contain F (final states of NFA).



Solⁿ : Transition Table - State

	0	1
$\rightarrow q_0$	q_0	q_1
q_1	$\{q_1, q_2\}$	q_1
* q_2	q_2	$\{q_1, q_2\}$

δ' transition for state q_0

$$\delta'(q_0, 0) = [q_0]$$

$$\delta'(q_0, 1) = [q_1]$$

δ' transition for state q_1

$$\delta'(q_1, 0) = [q_1, q_2] \rightarrow \text{new state generated}$$

$$\delta'(q_1, 1) = [q_1]$$

δ' transition for state q_2

$$\delta'(q_2, 0) = [q_2]$$

$$\delta'(q_2, 1) = [q_1, q_2]$$

δ' transition on $[q_1, q_2]$

$$\delta'([q_1, q_2], 0) = \delta(q_1, 0) \cup \delta(q_2, 0)$$

$$= \{q_1, q_2\} \cup \{q_2\}$$

$$= \{q_1, q_2\} = [q_1, q_2]$$

$$\delta'([q_1, q_2], 1) = \delta(q_1, 1) \cup \delta(q_2, 1)$$

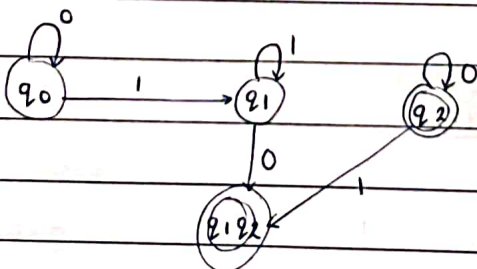
$$= \{q_1\} \cup \{q_1, q_2\}$$

$$= \{q_1, q_2\}$$

$$= [q_1, q_2]$$

The state $\{q_1, q_2\}$ is final state as well because it contains a final state q_2 .

state	0	1
$\rightarrow [q_0]$	$[q_0]$	$[q_1]$
$[q_1]$	$[q_1, q_2]$	$[q_1]$
$^* [q_2]$	$[q_2]$	$[q_1, q_2]$
$^* [q_1, q_2]$	$[q_1, q_2]$	$[q_1, q_2]$



★ Finite Automata with ϵ moves

NFA with ϵ move : If any FA contains ϵ transactions or move, the finite automata is called NFA with ϵ move.

ϵ closure : ϵ closure for any given state A means a set of states which can be reached from state A with only ϵ (null) move including the state A itself.

Steps for Converting NFA with ϵ to DFA

Step 1 : We will take ϵ closure for starting state of NFA as starting state of DFA.

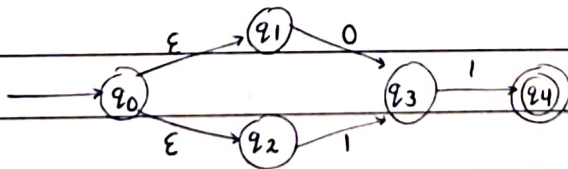
Step 2 : Find the states for each input symbol that can be traversed from the present. That means the union of transition value & their closures for each state of NFA present in the current state of DFA.

Step 3 : If we found a new state, take it as current state & repeat step 2.

Step 4 : Repeat step 2, 3 until there is no new state present in the transition table of DFA.

Step 5 : Mark the states of DFA as final state which contains the final state of NFA.

example :



ϵ closures : ϵ closure $\{q_0\} = \{q_0, q_1, q_2\}$

ϵ closure $\{q_1\} = \{q_1\}$

ϵ closure $\{q_2\} = \{q_2\}$

ϵ closure $\{q_3\} = \{q_3\}$

ϵ closure $\{q_4\} = \{q_4\}$

Let ϵ closure $\{q_0\} = \{q_0, q_1, q_2\}$ be state A.

$$\begin{aligned}
 \delta'(A, 0) &= \epsilon \text{ closure } \{ \delta((q_0, q_1, q_2), 0) \} \\
 &= \epsilon \text{ closure } \{ \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \} \\
 &= \epsilon \text{ closure } \{ q_3 \cup q_3 \cup q_3 \} \\
 &= \epsilon \text{ closure } \{ q_3 \} \\
 &= \{ q_3 \}
 \end{aligned}$$

Let q_3 be state B.

$$\begin{aligned}
 \delta'(A, 1) &= \epsilon \text{ closure } \{ \delta((q_0, q_1, q_2), 1) \} \\
 &= \epsilon \text{ closure } \{ \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \} \\
 &= \epsilon \text{ closure } \{ q_3 \} \\
 &= \{ q_3 \}
 \end{aligned}$$

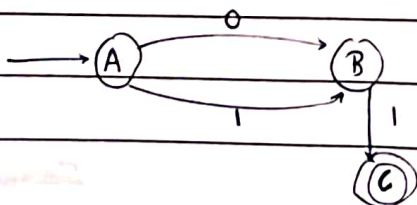
$$\begin{aligned}
 \delta'(B, 0) &= \epsilon \text{ closure } \{ \delta(q_3, 0) \} \\
 &= \epsilon \text{ closure } \{ \emptyset \} \\
 &= \emptyset
 \end{aligned}$$

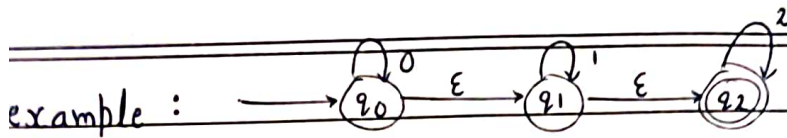
$$\begin{aligned}
 \delta'(B, 1) &= \epsilon \text{ closure } \{ \delta(q_3, 1) \} \\
 &= \epsilon \text{ closure } \{ q_4 \} \\
 &= \{ q_4 \}
 \end{aligned}$$

Let q_4 be state C.

$$\begin{aligned}
 \delta'(C, 0) &= \epsilon \text{ closure } \{ \delta(q_4, 0) \} \\
 &= \emptyset
 \end{aligned}$$

$$\begin{aligned}
 \delta'(C, 1) &= \epsilon \text{ closure } \{ \delta(q_4, 1) \} \\
 &= \emptyset
 \end{aligned}$$





$$\epsilon \text{ closure } \{q_0\} = \{q_0, q_1, q_2\}$$

$$\epsilon \text{ closure } \{q_1\} = \{q_1, q_2\}$$

$$\epsilon \text{ closure } \{q_2\} = \{q_2\}$$

Let $\epsilon \text{ closure } \{q_0\} = \{q_0, q_1, q_2\}$ be state A.

$$\begin{aligned} \delta'(A, 0) &= \epsilon \text{ closure } \{ \delta((q_0, q_1, q_2), 0) \} \\ &= \epsilon \text{ closure } \{ \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \} \\ &= \epsilon \text{ closure } \{ q_0 \cup \phi \cup \phi \} = \epsilon \text{ closure } \{ q_0 \} \\ &= \{ q_0, q_1, q_2 \} \end{aligned}$$

$$\begin{aligned} \delta'(A, 1) &= \epsilon \text{ closure } \{ \delta((q_0, q_1, q_2), 1) \} \\ &= \epsilon \text{ closure } \{ \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \} \\ &= \epsilon \text{ closure } \{ \phi \cup q_1 \cup \phi \} \\ &= \epsilon \text{ closure } \{ q_1 \} \\ &= \{ q_1, q_2 \} \quad \text{call it as state B} \end{aligned}$$

$$\begin{aligned} \delta'(A, 2) &= \epsilon \text{ closure } \{ \delta((q_0, q_1, q_2), 2) \} \\ &= \epsilon \text{ closure } \{ \delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2) \} \\ &= \epsilon \text{ closure } \{ \phi \cup \phi \cup q_2 \} \\ &= \epsilon \text{ closure } \{ q_2 \} \\ &= \{ q_2 \} \quad \text{call it as state C.} \end{aligned}$$

$$\begin{aligned} \delta'(B, 0) &= \epsilon \text{ closure } \{ \delta((q_1, q_2), 0) \} \\ &= \epsilon \text{ closure } \{ \phi \} = \phi \end{aligned}$$

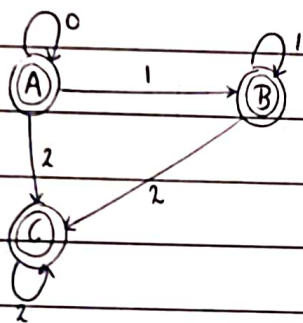
$$\begin{aligned} \delta'(B, 1) &= \epsilon \text{ closure } \{ \delta((q_1, q_2), 1) \} \\ &= \epsilon \text{ closure } \{ q_1 \} \\ &= \{ q_1, q_2 \} \end{aligned}$$

$$\begin{aligned}
 S'(B, 2) &= \epsilon \text{ closure } \{ \delta(q_1, q_2), 2 \} \\
 &= \epsilon \text{ closure } \{ q_2 \} \\
 &= \{ q_2 \}
 \end{aligned}$$

$$\begin{aligned}
 S'(C, 0) &= \epsilon \text{ closure } \{ \delta(q_2, 0) \} \\
 &= \emptyset
 \end{aligned}$$

$$\begin{aligned}
 S'(C, 1) &= \epsilon \text{ closure } \{ \delta(q_2, 1) \} \\
 &= \emptyset
 \end{aligned}$$

$$\begin{aligned}
 S'(C, 2) &= \epsilon \text{ closure } \{ \delta(q_2, 2) \} \\
 &= \{ q_2 \}
 \end{aligned}$$



Final state ' q_2 ' lies in all A, B, C
 $\therefore$ All are final states.

* Regular Expression

- The language accepted by finite automata can be easily described by simple expressions called regular expressions.
- The language accepted by some regular expression are referred to as regular languages.
- A regular expression can also be described as a sequence of pattern that defines a string.
- Regular expressions are used to match character combinations in strings.

If Σ is alphabet then regular expression over this can be described by the following rules :

In regular expression:

x^* means zero or more occurrences of x . It can generate $(\epsilon, x, xx, \dots)$

x^+ means one or more occurrences of x . It can generate $(x, xx, xxx, \dots)$

Date _____

i. Any symbol from $\Sigma, \emptyset, \epsilon$ are regular expressions

ii. If ' x_1 ' & ' x_2 ' are 2 regular expressions then union of these represented as $x_1 \cup x_2$ or $x_1 + x_2$ is also regular expression

iii. If ' x_1 ' & ' x_2 ' are 2 regular expressions, then concatenation of these as $x_1 \cdot x_2$ is also regular expression.

iv. Kleen closure of regular expression x is denoted by x^* is also regular expression.

Regular expression for language over $\Sigma = \{0, 1\}$

- Set representing $\{00\} = 00$

- Set representing all strings of 0's & 1's = $(0+1)^*$

- Set of all strings with atleast 2 consecutive 0s $(0+1)^* 00 (0+1)^*$

- Set of all strings ending in 011 $(0+1)^* 011$

- Set of all strings repeating any number of 0's followed by any number of 1's = $0^* 1^*$

Regular expression for language over $\Sigma = \{a, b\}$

- Set of all strings ending with either a or bb = $(a+b)^* (a+bb)$

- Set of all strings consisting of even numbers of a's followed by odd numbers of b's = $(aa)^* (bb)^* b$

- Set of strings representing even numbers of a's = $(b^* a b^* a b^*)^* + b^*$

- Starting with a but not have consecutive b's = $(a+ab)^*$

- Atleast one 'a' & atleast 'one b' = $(a+b)^* (ab+ba)(a+b)^*$

$[(a+b)^* a (a+b)^* b (a+b)^*] + [(a+b)^* b (a+b)^* a (a+b)^*]$

- Atleast two occurrences of b's between any two occurrences of a's = $(abbbb^* a)^*$ or $(b + (abbb^* a))^*$

- Every 'a' immediately followed by bb = $(abb + b)^*$

Two regular expression P & Q are equivalent if P & Q represent same set of strings.

$+$ $\rightarrow$ Union

$\epsilon, \wedge$ both are same

$\phi R \rightarrow$ Concatenation of ϕ & R

Date _____

Identities Related to Regular Expressions

- $\phi + R = R$
- $\phi R = R\phi = \phi$
- $\epsilon R = R\epsilon = R$
- $\epsilon^* = \epsilon$ & $\phi^* = \epsilon$
- $R + R = R$
- $R^* R^* = R^*$
- $RR^* = R^* R$
- $(R^*)^* = R^*$
- $\epsilon + RR^* = \epsilon + R^* R = R^*$
- $(PQ)^* P = P(QP)^*$
- $(P+Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$
- $(P+Q)R = PR + QR$
- $R(P+Q) = RP + RQ$

★ Arden's Theorem

Arden's theorem states that: "If P & Q are 2 regular expressions over Σ & if P doesn't contain ϵ , then following eqⁿ has unique solⁿ."

Equation - $R = Q + RP$

Solution - $R = QP^*$

Q $\rightarrow$ Prove $(1+011)^* = \epsilon + 1^*(011)^*(1^*(011)^*)^*$

Solⁿ: RHS $\epsilon + 1^*(011)^*(1^*(011)^*)^*$ $\because \epsilon + RR^* = R^*$
Here $R = (1^*(011)^*)^*$

$\therefore$ LHS = RHS

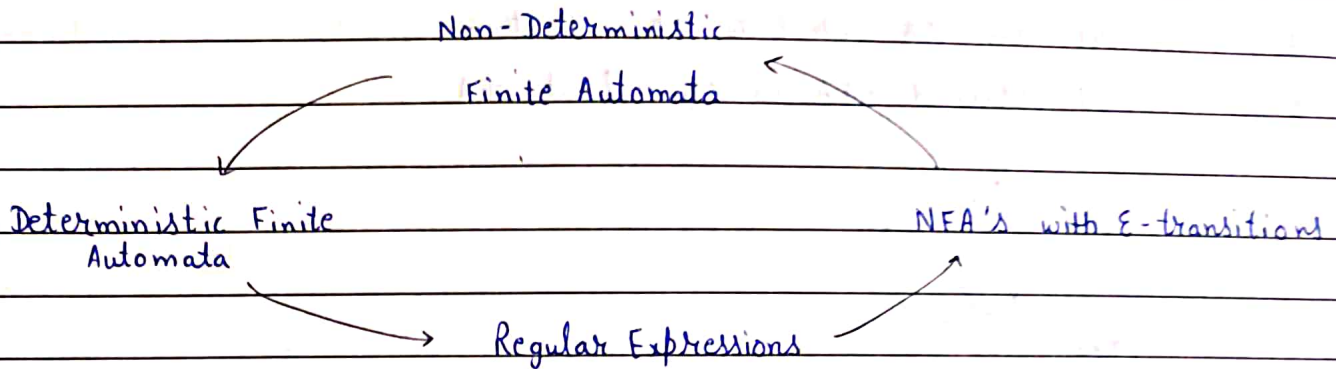
Q $\rightarrow$ Prove $(1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1) = 0^*1(0+10^*1)^*$
 $(1+00^*1) \left[\epsilon + (0+10^*1)^*(0+10^*1) \right]$
 $\epsilon + R^*R = R^*$

$(1+00^*1)(0+10^*1)^*$
 $1 \left(\underbrace{\epsilon + 00^*}_{\epsilon + RR^*} \right) (0+10^*1)^*$

$10^*(0+10^*1)^*$

Hence Proved.

* Equivalence of Finite Automata & Regular Expression



If x be regular expression, then there exists a NFA with ϵ moves which accepts $L(x)$

Construction of NFA with ϵ -moves :

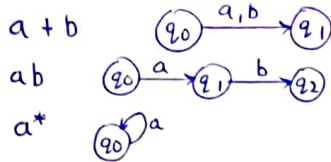
- Case 1 :
- i $\rightarrow x = \epsilon$ $\rightarrow$
 - ii $\rightarrow x = \phi$ $\rightarrow$
 - iii $\rightarrow x = a$ $\rightarrow$

Case 2 : i $\rightarrow$ Union ($x = x_1 + x_2$)

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, f_1)$ $M_2 = (Q_2, \Sigma, \delta_2, q_2, f_2)$

$L(M_1) = L(x_1)$ & $L(M_2) = L(x_2)$

If RE is in form $a + b$



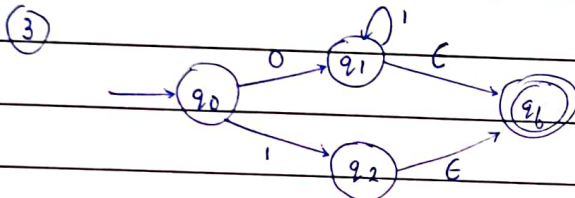
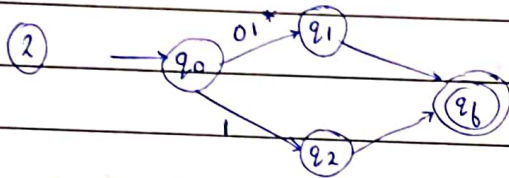
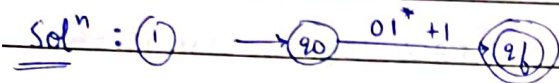
Date _____

Examples: Construct NFA by using regular expression $01^* + 1$

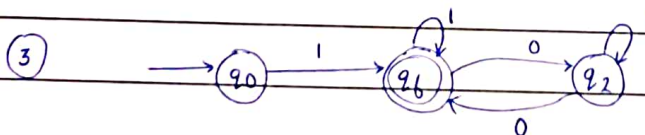
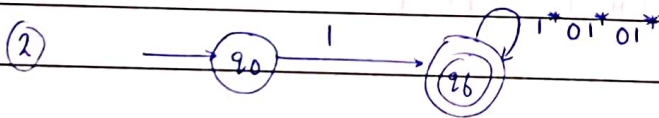
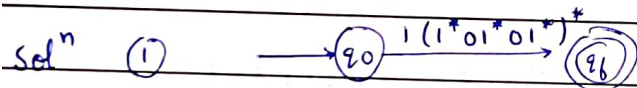
Step 1 - Make transition diagram for regular expression, using NFA with ϵ moves

Step 2 - Convert this NFA with ϵ to NFA without ϵ

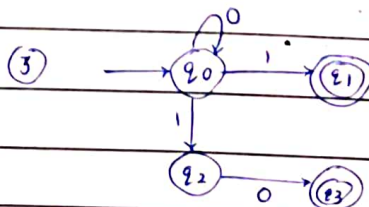
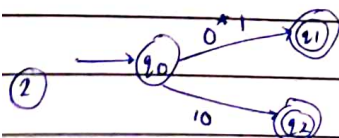
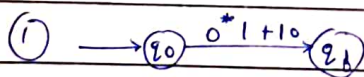
Step 3 - Convert obtained NFA to equivalent DFA



Example (2) Design NFA for RE $1(1^*01^*01^*)^*$

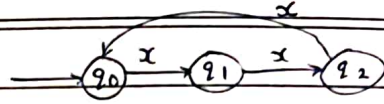


Example 3: FA of RE $0^*1 + 10$

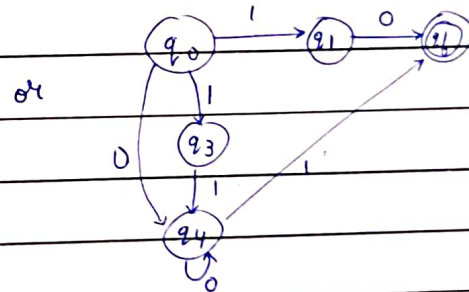
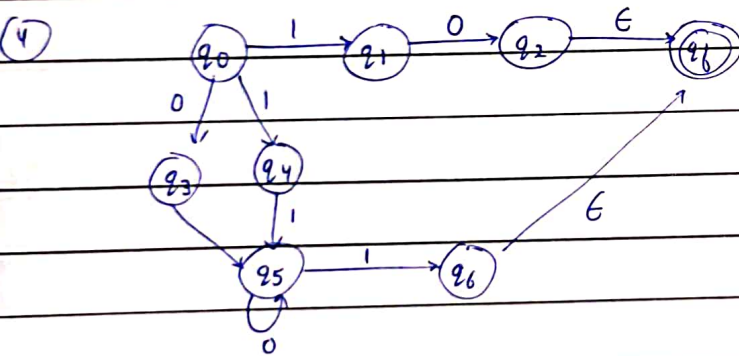
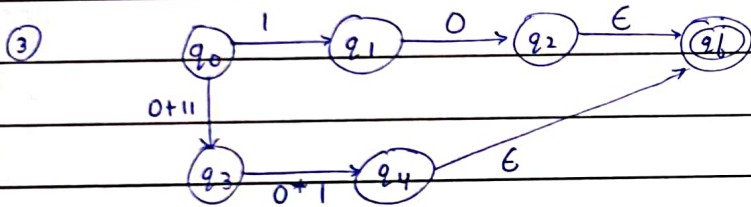
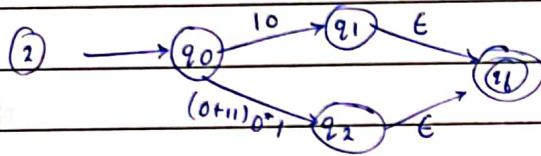
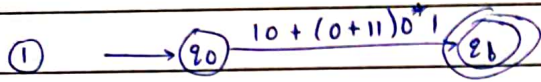


Date _____

example 4 $(xxx)^*$



example 5 : $10 + (0+11)0^*1$





Initialize G to set of maximally general hypothesis
 S to " specific

For each training example d

• If d is true example.

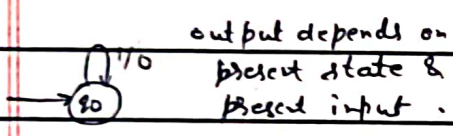
Remove from G any hypothesis inconsistent with d

For each hypothesis s in S that is inconsistent with d

- Remove s from S
- Add all minimal generalization h of s such that h is consistent with d , some members of G is more general than h .

• Remove from S any hypothesis that is more general than another hypothesis in S .

Mealy Machine

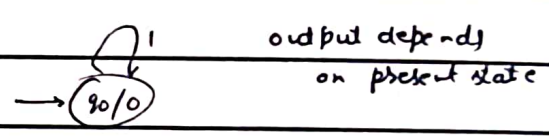


output depends on present state & present input.

- Less states are req.
- More hardware req. for circuit implementation.
- React slower to inputs
- Output $\rightarrow$ asynchronous
- Difficult to design.
- If input change, output change.

$$\lambda: Q \times \Sigma \rightarrow \Delta$$

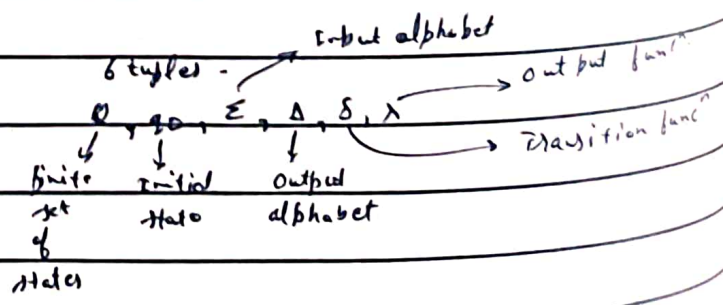
Mooore Machine.



output depends on present state

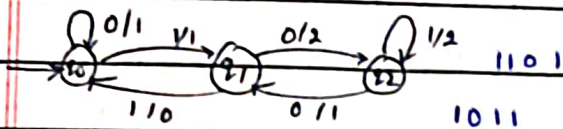
- More states are req.
- Less hardware req.
- Reacts faster to inputs.
- Output - synchronous to clock.
- Easy to design
- If input changes, output doesn't change.

$$\lambda: Q \rightarrow \Delta$$



Mealy Machine

- output depends on present state & present input.
- Input string length 'n' → output 'n'

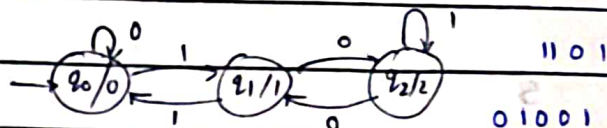


- Mealy output is asynchronous (faster)

$$\lambda : Q \times \Sigma \rightarrow \Delta$$

Moore Machine

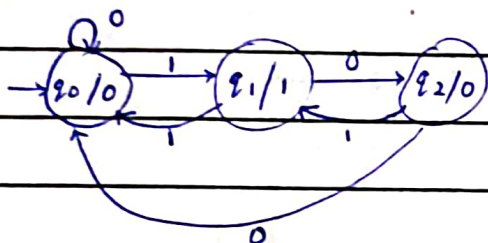
- outputs only depends on present state, independent on input
- Input string length 'n' → output string 'n+1'



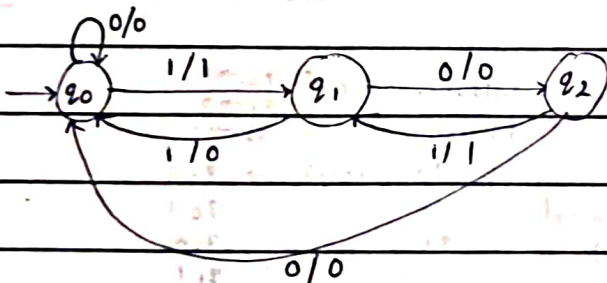
- Output is synchronous with clock (slower)

$$\lambda : Q \rightarrow \Delta$$

★ Moore Machine to Mealy Machine.



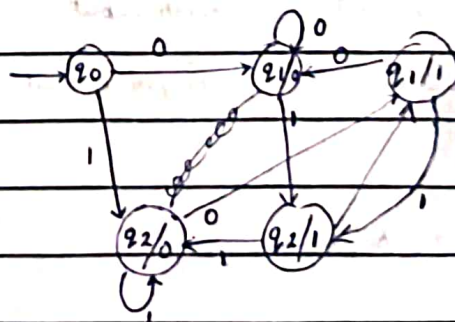
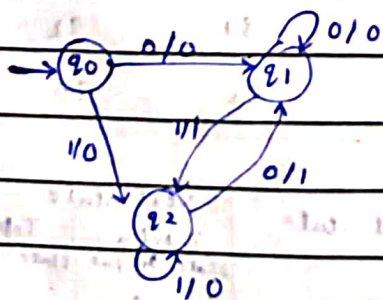
	next state		output
	0	1	
q0	q0	q1	0
q1	q2	q0	1
q2	q0	q1	0



Current	Next State	
	0	1
q0	q0, 0	q1, 1
q1	q2, 0	q0, 0
q2	q0, 0	q1, 1

m state, n output → Atmost m state.

★ Mealy to Moore Machine



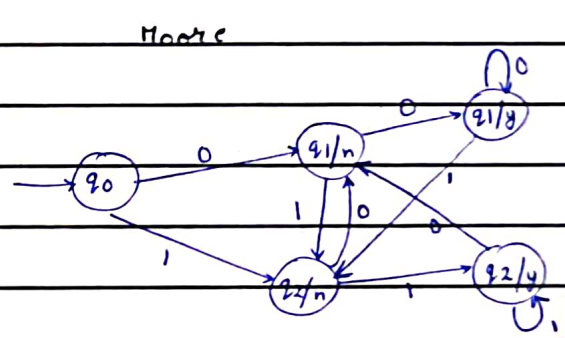
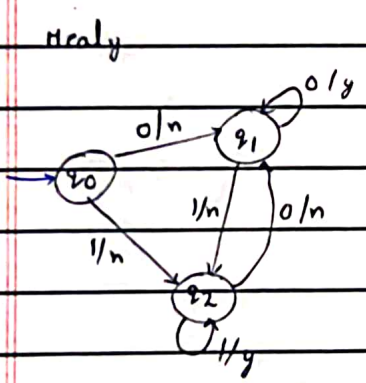
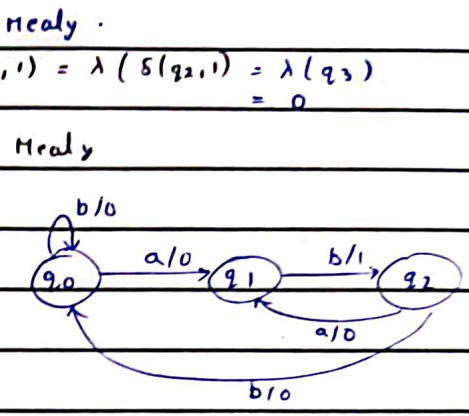
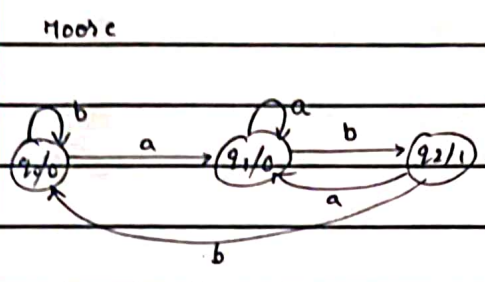
m state, n output → max. Total $m \times n$ state

Present State	Next State		output
	0	1	
q ₀	q ₃	q ₁	0
q ₁	q ₁	q ₂	1
q ₂	q ₂	q ₃	0
q ₃	q ₃	q ₀	0

Present State	Next State	
	0	1
q ₀	(q ₃ , 0)	(q ₁ , 1)
q ₁	(q ₁ , 1)	(q ₂ , 0)
q ₂	(q ₂ , 0)	(q ₃ , 0)
q ₃	(q ₃ , 0)	(q ₀ , 0)

DATE: __/__/__
PAGE: __

Moore $\lambda'(q, a) = \lambda(\delta(q, a))$ example $\lambda'(q_2, 1) = \lambda(\delta(q_2, 1)) = \lambda(q_3) = 0$



	0	1	output
q ₀	q ₁ n	q ₂ n	
q ₁ n	q ₁ y	q ₂ n	n
q ₁ y	q ₁ y	q ₂ n	y
q ₂ n	q ₁ n	q ₂ y	n
q ₂ y	q ₁ n	q ₂ y	y

